

Note: I only tested entities and classes that I wrote. While overall coverage is 64% in the Backend folder, I did tests for all my entities (and most classes).

Billing package

Package: com.questbuddy.billing

Class: ProductCheckoutController

Methods I wrote and test coverage:

Method name	Wrote test cases?
successEndpoint(long userId)	✓
cancelEndpoint()	✓
checkoutProducts(long userId, ProductRequest productRequest)	✓

Tests exercising these methods:

From ProductCheckoutControllerTest:

- **testSuccessEndpoint_userBecomesPremium()**
 - GET /success/5
 - Covers successEndpoint() (branch: user becomes premium)
 - Verifies userRepository.save() is called
- **testSuccessEndpoint_userAlreadyPremium()**
 - GET /success/5

- Covers successEndpoint() (branch: user already premium)
 - Verifies save() is *not* called
 - **testCancelEndpoint()**
 - GET /cancel
 - Covers cancelEndpoint()
 - **testCheckoutProducts_success()**
 - POST /checkout/premium/5
 - Covers checkoutProducts()
 - Mocks StripeService.checkout()
-

Class: ProductRequest

Methods I wrote and test coverage:

Method name	Wrote test cases?
ProductRequest()	✓
ProductRequest(Long amount, Long quantity, String productName, String currency)	✓
getAmount(), setAmount()	✓
getQuantity(), setQuantity()	✓
getProductName(), setProductName()	✓
getCurrency(), setCurrency()	✓

Tests exercising these methods:

From ProductRequestTest:

- **testGettersSetters()**
 - Covers all setter/getter pairs
 - **testConstructor()**
 - Covers full constructor and all getters
-

Class: StripeResponse

Methods I wrote and test coverage:

Method name	Wrote test cases?
StripeResponse()	✓
StripeResponse(String status, String msg, String sessionId, String sessionUrl)	✓
get/set Status	✓
get/set Message	✓
get/set SessionId	✓
get/set SessionUrl	✓

Tests exercising these methods:

From StripeResponseTest:

- **testGettersSetters()**
 - Covers all setter/getter pairs
- **testConstructor()**
 - Covers full constructor + getters

Class: StripeService

Methods I wrote and test coverage:

Method name	Wrote test cases?
checkout(ProductRequest req, long userId)	✓

Tests exercising these methods:

From StripeServiceTest:

- **testCheckout_success()**
 - Mocks Session.create()
 - Covers successful session creation branch
 - Verifies returned StripeResponse is built correctly
- **testCheckout_stripeException()**
 - Mocks Session.create() to throw CardException
 - Covers exception fallback branch
 - Verifies "error" response and null session fields

Perfect — here is your **copy-and-paste-ready full coverage report** for the **Calendar package** in the *exact same style* as your friend's sample.

This is formatted so you can drop it directly into Google Docs / Word.

Calendar package

Package: com.questbuddy.calendar

Class: EventController

Methods I wrote and test coverage:

Method name	Wrote test cases?
update(Long userId, Long id, EventUpdateDTO body)	✓
create(Long userId, EventCreateDTO body)	✓
list(Long userId, Instant from, Instant to)	✓
listAll(Long userId, Instant from, Instant to)	✓
listByUser(Long requesterId, Long userId, Instant from, Instant to)	✓
get(Long userId, Long id)	✓
ping()	✓
delete(Long userId, Long id)	✓
onValidation(ValidationException)	✓ (indirect)
onNotFound(EventService.ResourceNotFound)	✓ (indirect)
onBadRequest(IllegalArgumentException)	x

Tests exercising these methods:

From EventControllerTest:

- **testUpdate_success()**
 - PUT /events/10

- Covers update() (success branch)
- **testUpdate_validationException()**
 - Mocks service → throws ValidationException
 - Covers onValidation() exception handler
- **testUpdate_notFound()**
 - Mocks → throw ResourceNotFound
 - Covers onNotFound()
- **testCreate_success()**
 - POST /events
 - Covers create() (success branch)
- **testCreate_invalidBody()**
 - POST {} → 400
 - Covers validation → onBadRequest() is *not* hit (thrown by Spring before controller)
- **testList_success()**
 - GET /events
 - Covers list()
- **testListAll_success()**
 - GET /events/all with no params
 - Covers listAll() (no range branch)
- **testListAll_between()**
 - GET /events/all?from&to

- Covers listAll() → listAllBetween() branch
 - **testListByUser_success()**
 - GET /events/user/8
 - Covers listByUser()
 - **testListByUser_between()**
 - GET /events/user/8?from&to
 - Covers listByUserBetween()
 - **testGet_success()**
 - GET /events/15
 - Covers get() success branch
 - **testGet_notFound()**
 - Covers onNotFound()
 - **testPing()**
 - GET /ping
 - Covers ping()
 - **testDelete_success()**
 - DELETE /events/33
 - Covers delete() success
 - **testDelete_notFound()**
 - DELETE /events/33 with mock not found
 - Covers onNotFound() exception path
-

Class: EventMapper

Methods I wrote and test coverage:

Method name	Wrote test cases?
toEntity(Long userId, EventCreateDTO dto)	✓
applyUpdate(Event e, EventUpdateDTO dto)	✓
toDto(Event e)	✓

Tests exercising these methods:

From EventMapperTest:

- **testToEntity()**
 - Covers toEntity()
 - **testApplyUpdate_allFields()**
 - Covers full update branch
 - **testApplyUpdate_nullFields_doNotChange()**
 - Covers null-handling branch
 - **testToDto()**
 - Covers toDto()
-

Class: EventService

Methods I wrote and test coverage:

Method name	Wrote test cases?
-------------	-------------------

<code>create(Long userId, EventCreateDTO dto)</code>	✓
<code>list(Long userId, Instant from, Instant to)</code>	✓
<code>listAll()</code>	✓
<code>listAllBetween(Instant from, Instant to)</code>	✓
<code>listByUser(Long userId)</code>	✓
<code>listByUserBetween(Long userId, Instant from, Instant to)</code>	✓
<code>get(Long userId, Long id)</code>	✓
<code>update(Long userId, Long id, EventUpdateDTO dto)</code>	✓
<code>delete(Long userId, Long id)</code>	✓
<code>checkRange(Instant start, Instant end) (private)</code>	✓ (indirect)

Tests exercising these methods:

From `EventServiceTest`:

`create()`

- `testCreate_success()` → success branch
- `testCreate_invalidRange()` → invalid range → `ValidationException`
- `testCreate_notificationFailsButStillSucceeds()` → notification exception branch

`list()`

- `testList_noRange()` → no range branch
- `testList_partialRangeInvalid()` → `ValidationException`
- `testList_validRange()` → valid range branch

`listAll()`

- `testListAll()` → covers `listAll()`

listAllBetween()

- **testListAllBetween_success()**
- **testListAllBetween_invalidRange()**

listByUser()

- **testListByUser()**

listByUserBetween()

- **testListByUserBetween_success()**
- **testListByUserBetween_invalidRange()**

get()

- **testGet_found()**
- **testGet_notFound()** → ResourceNotFound

update()

- **testUpdate_success()**
- **testUpdate_notFound()**
- **testUpdate_invalidRange()**
- **testUpdate_notificationFails()**

delete()

- **testDelete_success()**
- **testDelete_notFoundBeforeDeletion()**

- `testDelete_notFoundAfterDeletion()`

Class: Event (Entity)

Methods I wrote and test coverage:

Method name	Wrote test cases?
Getters/setters (id, userId, title, desc, start/end, location, allDay, createdAt, updatedAt)	<i>x</i>
@PrePersist onCreate()	<i>x</i>
@PreUpdate onUpdate()	<i>x</i>

Tests exercising this class:

No direct tests.

It is used indirectly through EventMapper and EventService; however, the **entity methods themselves are not tested**.

Class: DTOs: EventCreateDTO, EventUpdateDTO, EventResponseDTO

Methods I wrote and test coverage:

DTO	Wrote test cases?
EventCreateDTO	<i>x</i>
EventUpdateDTO	<i>x</i>
EventResponseDTO	<i>x</i>

Below is your **copy-and-paste-ready** report in the exact style as before (no emojis, minimal explanation, clean tables).

Trip Events package

Package: com.questbuddy.events.trip

Class: TripEventController

Methods I wrote and test coverage:

Method name	Wrote test cases?
edit(Long me, Long tripId, Long eventId, TripEventEditDTO in)	✓
create(Long me, Long tripId, TripEventCreateDTO in)	✓
list(Long me, Long tripId, Instant from, Instant to, int page, int size)	✓
delete(Long me, Long tripId, Long eventId)	✓

Tests exercising these methods (from TripEventControllerTest):

- testEdit_success
Covers edit(), verifies service call and success response.
- testCreate_success
Covers create(), verifies service call.

- testList_success
Covers list() with no range.
 - testList_withRange
Covers list() with from/to, page, size params.
 - testDelete_success
Covers delete() and verifies service call.
-

Class: TripEventManager

Methods I wrote and test coverage:

Method name	Wrote test cases?
fromCreate(Long tripId, Long creatorId, TripEventCreateDTO in)	✓
toDTO(TripEvent e)	✓
applyEdit(TripEvent e, TripEventEditDTO in)	✓
toJson(List refs)	✓
fromJson(String json)	✓

Tests exercising these methods (from TripEventManagerTest):

- testFromCreate_basic
Covers fromCreate() normal behavior.

- testFromCreate_nullAttachments
Covers attachments=null branch.
- testToDTO_basic
Covers toDTO(), including parsing attachments.
- testToDTO_deleted
Covers deleted=true branch.
- testApplyEdit_updatesOnlyNonNull
Covers partial update handling.
- testToJson_nullOrEmpty
Covers toJson() for null/empty.
- testToJson_valid
Covers serialization.
- testFromJson_nullOrBlank
Covers parsing empty/blank.
- testFromJson_valid
Covers parsing valid json.

Class: TripEventService

Methods I wrote and test coverage:

Method name	Wrote test cases?
list(Long me, Long tripId, Instant from, Instant to, int page, int size)	✓
create(Long me, Long tripId, TripEventCreatedDTO in)	✓

edit(Long me, Long tripId, Long eventId, TripEventEditDTO in)	✓
delete(Long me, Long tripId, Long eventId)	✓
validateDates(Instant start, Instant end) (private)	✓ (indirect)
ensureOwnerOrCreator(Long me, Long tripId, TripEvent e) (private)	✓ (indirect)
mirrorTripEventToCalendars(...) (private)	✓ (indirect through create)

Tests exercising these methods (from TripEventServiceTest):

List()

- testList_noRange
 - testList_withRange
- Both cover membership checks and range behavior.

Create()

- testCreate_success
Covers saving, notifications, membership, and calendar mirroring.
- testCreate_invalidDate
Covers validateDates error branch.

Edit()

- testEdit_success_creator
Covers editing by creator.

- testEdit_success_owner
Covers editing by owner.
- testEdit_notOwnerNotCreator
Covers forbidden branch.
- testEdit_eventNotFound
Covers event not found.

Delete()

- testDelete_success_creator
Covers deletion path, notifications, membership check.
- testDelete_forbidden
Covers forbidden delete.
- testDelete_eventNotFound
Covers event not found.

Class: TripEvent (Entity)

Methods I wrote and test coverage:

Method name	Wrote test cases?
-------------	-------------------

Getters/setters	X
-----------------	---

prePersist()	X
--------------	---

preUpdate()	X
-------------	---

isDeleted() ✓ (indirect via TripEventMapperTest.testToDTO_deleted)

Entity is used extensively but not directly unit tested, which is normal.

DTOs: TripEventCreateDTO, TripEventEditDTO, TripEventResponseDTO

Methods I wrote and test coverage:

DTO	Wrote test cases?
TripEventCreateDTO	✓ (indirect via mapper/service tests)
TripEventEditDTO	✓ (indirect via mapper/service tests)
TripEventResponseDTO	✓ (indirect via mapper tests)

No direct DTO tests needed since they are immutable records.

DIRECT MESSAGE MODULE

DirectMessageController

Method	Tested?
DirectMessageResponseDTO edit(Long me, Long peerId, Long messageId, DirectMessageEditDTO in)	✓
DirectMessageResponseDTO post(Long me, Long peerId, DirectMessageCreatedDTO in)	✓
Map<String,Integer> react(Long me, Long peerId, Long messageId, String emoji)	✓
void markRead(Long me, Long peerId, Long messageId)	✓
List<DirectMessageResponseDTO> list(Long me, Long peerId, Long beforeId, int limit)	✓
String ping()	✓
void delete(Long me, Long peerId, Long messageId)	✗ (controller delete endpoint NOT covered — service delete is tested, but controller route is NOT)
Map<String,Integer> unreact(Long me, Long peerId, Long messageId, String emoji)	✓

DirectMessageService

Method	Tested ?
List<DirectMessageResponseDTO> list(Long me, Long peerId, Long beforeId, int limit)	✓
DirectMessageResponseDTO post(Long me, Long peerId, DirectMessageCreateDTO in)	✓
DirectMessageResponseDTO edit(Long me, Long peerId, Long messageId, DirectMessageEditDTO in)	✓
void delete(Long me, Long peerId, Long messageId)	✓
Map<String,Integer> toggleReaction(Long me, Long peerId, Long messageId, String emoji)	✓
void markRead(Long me, Long peerId, Long messageId)	✓

DirectMessageMapper

Method	Tested ?
--------	-------------

DirectMessage toEntity(Long me, Long peerId, DirectMessageCreatedDTO in, Instant savedAt)	✓
void applyEdit(DirectMessage target, DirectMessageEditDTO in, Instant editedAt)	✓
DirectMessageResponseDTO toResponse(DirectMessage m, Map<String,Integer> reactionCounts, Set<String> myReactions)	✓
List<DirectMessageResponseDTO> toResponses(List<DirectMessage> messages, Map<Long,Map<String,Integer>> counts, Map<Long,Set<String>> mine)	✓

DirectMessage Model

(Models normally aren't unit-tested except through service tests)

Item	Tested?
Getters / setters	x
JPA behavior	x
Read receipt fields covered indirectly through service tests	✓ (indirectly via service)

DirectMessageReaction Model

Item	Tested ?
Getters / setters	<i>x</i>
Entity mapping	<i>x</i>

TRIP MESSAGE MODULE

TripMessageController

Method	Tested ?
TripMessageResponseDTO edit(Long me, Long tripId, Long messageId, TripMessageEditDTO in)	✓
TripMessageResponseDTO post(Long me, Long tripId, TripMessageCreatedDTO in)	✓
Map<String,Integer> react(Long me, Long tripId, Long messageId, Map<String,String> body)	✓
List<TripMessageResponseDTO> list(Long me, Long tripId, Long beforeId, int limit)	✓
String ping()	✓
TripMessageResponseDTO delete(Long me, Long tripId, Long messageId, Long version)	✓
Map<String,Integer> unreact(Long me, Long tripId, Long messageId, String emoji)	✓

TripMessageService

Method	Tested ?
TripMessageResponseDTO post(Long me, Long tripId, TripMessageCreatedDTO in)	✓
List<TripMessageResponseDTO> list(Long me, Long tripId, Long beforeId, int limit)	✓
TripMessageResponseDTO edit(Long me, Long tripId, Long messageId, TripMessageEditDTO in)	✓
TripMessageResponseDTO delete(Long me, Long tripId, Long messageId, Long version)	✓
Map<String,Integer> toggleReaction(Long me, Long tripId, Long messageId, String emoji)	✓
int markReadProgress(Long me, Long tripId, Long upToMessageId)	✓

TripMessageMapper

Method	Tested ?
TripMessage toEntity(TripMessageCreatedDTO in, long tripId, long senderId, Instant sentAtFallback, Instant savedAt)	✓

TripMessageResponseDTO toResponse(TripMessage m, Map<String,Integer> reactionCounts, Set<String> myReactions)	✓
---	---

void applyEdit(TripMessage target, TripMessageEditDTO in, Instant editedAt)	✓
---	---

List<TripMessageResponseDTO> toResponses(List<TripMessage> messages, Map<Long,Map<String,Integer>> counts, Map<Long,Set<String>> mine)	✓
--	---

TripMessage Model

Item	Tested?
Getters/setters, JPA entity	✗
Versioning logic (optimistic lock)	Covered indirectly via service tests ✓

MessageReaction Model (Trip)

Item	Tested ?
Getters/setters	✗

TripReadProgress Model

Item	Tested?
Used in markReadProgress tests indirectly	✓ (indirect)
Getters/setters	x

WEBSOCKET MODULE

ChatBroadcaster

Method	Tested ?
tripMessageNew(Long tripId, TripMessageResponseDTO dto)	✓
tripEdit(Long tripId, TripMessageResponseDTO dto)	✓
tripDelete(Long tripId, TripMessageResponseDTO dto)	✓
tripReactionToggle(Long tripId, Long messageId, String emoji)	✓
tripReadReceipt(Long tripId, Long readerId, Long upToMessageId, Integer unreadCount)	✓
dmMessageNew(long u1, long u2, DirectMessageResponseDTO dto)	✗
dmEdit(long u1, long u2, DirectMessageResponseDTO dto)	✗
dmDelete(long u1, long u2, DirectMessageResponseDTO dto)	✗
dmReactionToggle(long u1, long u2, Long messageId, String emoji)	✗

dmReadReceipt(long u1, long u2, Long upToMessageId, Long readerId, Integer unreadCount)	X
---	---

(ChatBroadcaster tests only covered TRIP methods, not DM ones.)

DirectChatEndpoint (WebSocket)

(All public methods shown as they are WebSocket lifecycle handlers)

Method	Tested ?
onOpen(Session, String, Long)	X
onMessage(String, Session, String, Long)	X
onClose(Session, String, Long)	X
onError(Session, Throwable, String, Long)	X
sendToPair(String pairKey, String json)	X
canonicalPairKey(long u1, long u2)	X

(WebSocket endpoints are almost never unit-tested; normal.)

TripChatEndpoint (WebSocket)

Method	Tested ?
onOpen(Session, Long, Long)	X
onMessage(String, Session, Long, Long)	X
onClose(Session, Long, Long)	X
onError(Session, Throwable, Long, Long)	X
sendToTrip(Long, String)	X

WsPing

Method	Tested ?
o(Session)	X

Notification Module Coverage

Class: NotificationController

Methods and test coverage

Method	Tested ?
ResponseEntity<NotificationResponseDTO> markRead(Long id, Long me)	✓
ResponseEntity<NotificationResponseDTO> create(NotificationCreatedDTO dto)	✓
String ping()	✓
ResponseEntity<List<NotificationResponseDTO>> list(Long me, Boolean unread)	✓
ResponseEntity<Void> delete(Long id, Long me)	✓

Tests exercising these methods (from NotificationControllerTest):

- testPing
- testMarkRead_success
- testMarkRead_notFound
- testMarkRead_badRequest
- testMarkRead_forbidden

- testCreate_success
 - testCreate_badReference
 - testList_all
 - testList_unreadOnly
 - testDelete_success
 - testDelete_notFound
 - testDelete_badRequest
 - testDelete_forbidden
-

Class: NotificationService

Public methods and test coverage

Method	Tested ?
Notification create(NotificationCreatedDTO dto)	✓
List<Notification> listForUser(Long recipientId, Boolean unreadOnly)	✓
Notification markRead(Long id, Long recipientId)	✓
long countUnread(Long recipientId)	x

boolean deleteForOwner(Long id, Long recipientId)

✓

Tests exercising these methods (from NotificationServiceTest):

Create():

- testCreate_success
- testCreate_recipientNotFound
- testCreate_tripNotFound
- testCreate_eventNotFound
- testCreate_taskNotFound
- testCreate_badReference

List:

- testListForUser_unreadOnly
- testListForUser_all

Mark read:

- testMarkRead_notFound
- testMarkRead_wrongOwner
- testMarkRead_alreadyRead
- testMarkRead_newlyRead

Delete:

- testDelete_success
- testDelete_notFound

- testDelete_forbidden

Note: countUnread() has no direct unit test.

Class: NotificationMapper

Methods and coverage

Method	Tested ?
Notification fromCreate(NotificationCreatedDTO, User, Trip, Event, Task)	✓
NotificationResponseDTO toResponse(Notification)	✓

Tests (from NotificationMapperTest):

- testFromCreate_defaultsToReminder
 - testToResponse_handlesNullReferences
-

Class: Notification (JPA Entity)

Coverage summary

Feature	Tested?
---------	---------

Getters / setters	X
-------------------	---

JPA relationships	X
-------------------	---

Automatic timestamp (createdAt)	X
---------------------------------	---

isRead flag logic	Covered indirectly via NotificationServiceTest
-------------------	--

Entities typically are not directly unit tested.

Class: NotificationRepository

Coverage summary

Method	Tested ?
Spring Data JPA query methods	X

Repositories are not unit tested (normal). They are exercised indirectly through service tests.

Class: NotificationBroadcaster

Public methods and test coverage

Method	Tested?
publishNew(Notification n)	✓
publishRead(Long recipientId, Long notificationId, long unreadCount)	✗ (no direct test)

Notes

NotificationBroadcasterTest covers only the behavior of publishNew.
publishRead is used in NotificationServiceTest, but not directly asserted or intercepted via the websocket mock.
The private send() helper is not considered (private).

Class: NotificationEndpoint (WebSocket)

Public methods and test coverage

Method	Tested?
onOpen(Session, Long)	✗
onMessage(String, Session, Long)	✗
onClose(Session, Long)	✗

onError(Session, Throwable, Long)	X
-----------------------------------	---

static sendToUser(Long, String)	✓ (indirect via NotificationBroadcasterTest validating call invocation)
---------------------------------	---

Note: sendToUser is invoked through mocked static calls in NotificationBroadcasterTest.

Class: NotificationType (Enum)

Coverage summary

Feature	Tested ?
Enum constants	X

Enums generally don't require testing.

Trip Module Coverage

Class: TripController

Methods and test coverage

Method	Tested ?
TripResponseDTO update(Long userId, Long id, TripUpdateDTO body)	✓
TripResponseDTO create(Long userId, TripCreateDTO body)	✓
List<TripResponseDTO> list(Long userId)	✓
TripResponseDTO get(Long userId, Long id)	✓
void delete(Long userId, Long id)	✓

Tests exercising these methods (TripControllerTest):

- update_success_returnsDto
- create_success_returnsDto
- list_returnsTrips
- get_success_returnsDto
- delete_success_returns204

Class: TripService

Public methods and test coverage

Method	Tested ?
TripResponseDTO create(Long ownerId, TripCreatedDTO dto)	✓
List<TripResponseDTO> list(Long userId)	✓
TripResponseDTO get(Long ownerId, Long id)	✓
void delete(Long ownerId, Long id)	✓
TripResponseDTO update(Long ownerId, Long id, TripUpdatedDTO dto)	✓

Internal helper logic tested indirectly

- checkRange(LocalDate start, LocalDate end) → tested via create_throwsWhenEndDateBeforeStartDate and update_invalidRange_throws
- checkStartPoint(Double lat, Double lon) → indirectly exercised through create() and update() success paths
- Membership seeding (seedOwner) → tested in create_success_savesTrip_andSeedsOwner

Tests (TripServiceTest):

Create:

- create_success_savesTrip_andSeedsOwner
- create_throwsWhenEndDateBeforeStartDate

List:

- list_combinesOwnedAndMemberTrips_sorted_noDuplicates
- list_emptyOwnedAndMember_returnsEmptyList

Get:

- get_success_returnsDto

- get_notFound_throws

Delete:

- delete_success_callsRepo
- delete_zeroDeleted_throwsNotFound

Update:

- update_success_savesAndReturnsDto
- update_notFound_throws
- update_invalidRange_throws

Class: TripMapper

Public methods and test coverage

Method	Tested ?
Trip toEntity(Long ownerId, TripCreatedDTO dto)	✓
TripResponseDTO toDto(Trip t)	✓
void applyUpdate(Trip t, TripUpdateDTO dto)	✓

Tests (TripMapperTest):

- toEntity_mapsFieldsCorrectly
 - toDto_mapsFieldsCorrectly
 - applyUpdate_updatesOnlyProvidedFields
-

Class: Trip (JPA Entity)

Public API and test coverage

Feature	Tested ?
Getters / setters	✓
onInsert()	✓
onUpdate()	✓
Timestamp behavior	✓
Null latitude/longitude	✓

Tests (TripTest):

- prePersist_setsCreatedAtAndUpdatedAt
- preUpdate_updatesUpdatedAtOnly
- setters_and_getters_work
- setting_null_lat_lon_works

Class: TripRepository

Coverage summary

Method	Tested?
findAllByOwnerId(Long, Sort)	✓ (indirect via service tests)
findAllForUserAsMemberWithStatus(Long, Status)	✓ (indirect)
findByIdAndOwnerId(Long, Long)	✓ (indirect)

deleteByIdAndOwnerId(Long, Long)

✓ (indirect)

Repositories are tested indirectly through TripServiceTest.

Class: TripCreateDTO / TripUpdateDTO / TripResponseDTO

Coverage summary

DTO	Tested?
TripCreateDTO	✓ (indirectly exercised in create tests and mapper tests)
TripUpdateDTO	✓ (indirectly via update and applyUpdate tests)
TripResponseDTO	✓ (returned and asserted in controller + service tests)

DTOs generally not directly unit tested.

TripMember Module Coverage

Class: TripInviteController

Public methods

Method	Tested ?
List<PendingInviteDTO> listPendingForUser(Long me, Long userId)	✓

Tests covering these:

- testListPendingSuccess
 - testListPendingForbidden
 - testListPendingUserNotFound
-

Class: TripMemberController

Public methods

Method	Tested?
ResponseEntity<Void> approve(Long me, Long tripId, UpdateStatusDTO body)	✓
ResponseEntity<Void> decline(Long me, Long tripId, UpdateStatusDTO body)	✓ (validation branch tested)
ResponseEntity<Void> invite(Long me, Long tripId, InviteDTO body)	✓
List<UserSummaryDTO> list(Long me, Long tripId)	✓

ResponseEntity<Void> remove(Long me, Long tripld, Long userId) ✓

ResponseEntity<Void> leave(Long me, Long tripld) ✓

Tests:

Approve:

- testApproveSuccess
- testApproveBadStatus400
- testApproveNotFound

Decline:

- testDeclineBadStatus400

Invite:

- testInviteSuccess
- testInviteForbidden
- testInviteNotFound

List:

- testListMembersSuccess
- testListMembersForbidden
- testListMembersNotFound

Remove:

- testRemoveSuccess
- testRemoveForbidden

- testRemoveNotFound

Leave:

- testLeaveSuccess
- testLeaveNotFound

Class: TripMembershipService

Public methods

Method	Tested ?
invite(Long inviterId, Long tripId, Long userId)	✓
approve(Long approverId, Long tripId)	✓
decline(Long userId, Long tripId)	✓
remove(Long actorId, Long tripId, Long targetUserId)	✓
List<User> listAccepted(Long requesterId, Long tripId)	✓
seedOwner(Long ownerId, Long tripId)	✓
List<TripMember> listPendingInvitesForUser(Long requesterId, Long targetUserId)	✓
void ensureMember(Long userId, Long tripId)	✓
boolean isOwner(Long userId, Long tripId)	✓

Tests:

Invite:

- testInviteCallerNotOwner403
- testInviteTripNotFound

- testInviteInviteeNotFound
- testInviteNoOpIfAlreadyExists
- testInviteSuccess

Approve:

- testApproveNotFound404
- testApproveAlreadyAcceptedNoOp
- testApproveSuccess

Decline:

- testDeclineNotFound404
- testDeclineAlreadyAcceptedNoOp
- testDeclinePendingDeletes

Remove:

- testRemoveForbidden
- testRemoveSelf
- testRemoveByOwner

List accepted:

- testListAcceptedForbidden
- testListAcceptedSuccess

Seed owner:

- testSeedOwnerAlreadyExistsNoOp

- testSeedOwnerTripNotFound
- testSeedOwnerSuccess

List pending:

- testListPendingForbidden
- testListPendingUserNotFound
- testListPendingSuccess

Misc:

- testEnsureMemberForbidden
- testEnsureMemberSuccess
- testIsOwnerDelegatesToGate

Class: PendingInviteDTO

Public static factory

Method	Tested ?
static PendingInviteDTO from(TripMember m)	✓

Tests:

- testFromWithFullData
 - testFromWithNulls
 - testTripLabelFallbacks
-

Class: InviteDTO

Methods

Method	Tested ?
record accessor: userId()	✓

Test:

- testInviteDTOStoresValue
-

Class: UpdateStatusDTO

Public API

Method	Tested ?
record accessor: status()	✓
Validation annotations (@NotNull, @Pattern("ACCEPTED"))	✓

Tests:

- testValidAcceptedStatus
 - testInvalidStatusRejected
 - testNullStatusRejected
-

Class: UserSummaryDTO

Public static factory

Method	Tested ?
static UserSummaryDTO from(User u)	✓

Tests:

- testFromWithFullName
 - testFromWithFirstOnly
 - testFromWithLastOnly
 - testFromWithUsernameFallback
-

Class: TripMember (entity)

Public API

Feature	Tested ?
Getters / setters	✓
onCreate()	✓
onUpdate()	✓

Tests:

- testPrePersistSetsTimestamps
 - testPreUpdateUpdatesTimestamp
 - testGettersAndSetters
-

Class: TripMembershipGateJPA

Public methods

Method	Tested?
boolean isMember(Long tripId, Long userId)	Not directly (indirect via service listAccepted, ensureMember)
boolean isOwner(Long tripId, Long userId)	Not directly (indirect via service.isOwner & service.gate delegation)

Status:

Indirectly covered through TripMembershipServiceTest (e.g., isOwner delegations, gate.isMember checks).

Repository behavior not unit tested, which is normal.

Class: TripMemberRepository

Coverage

Method	Tested?
Custom finders & delete ops	✓ (indirect)

Indirectly exercised heavily in TripMembershipServiceTest.

User Module Coverage

Class: AuthController

Public methods

Method	Tested ?
ResponseEntity<?> login(LoginRequest body)	✓
ResponseEntity<?> signup(SignupRequest body)	✓
ResponseEntity<?> onIntegrity(DataIntegrityViolationException e)	✓

Tests exercising these:

Login:

- login_missingFields_returns400
- login_invalidCredentials_returns401
- login_success_returnsUserId

Signup:

- signup_missingFields_returns400
- signup_emailExists_returns409
- signup_usernameExists_returns409
- signup_success_createsUserAndReturns201

Exception handler:

- onIntegrity_returns409WithConstraintBody

Class: UserController

Public methods

Method	Tested?
ResponseEntity<?> update(Long id, Long userId, UpdateProfileRequest body)	✓
ResponseEntity<?> updateMe(Long userId, UpdateProfileRequest body)	✓
ResponseEntity<List<UserDto>> signupBatch(List<SignupRequest> bodies)	✓
ResponseEntity<UserDto> get(Long id)	✓
ResponseEntity<?> me(Long userId)	✓
ResponseEntity<List<UserDto>> listPremiumUsers()	✓ (commented-out test exists, but method indirectly exercised since mapping logic tested in UserService tests)

Tests:

Update:

- update_missingUserHeader_returns401
- update_forbiddenWhenHeaderDoesNotMatchPathId
- update_success_returnsUpdatedDto

UpdateMe:

- updateMe_missingHeader_returns401
- updateMe_success_updatesCurrentUser

Signup batch:

- signupBatch_createsOnlyEntriesWithRequiredFields

Get user:

- getUser_found_returnsDto
- getUser_notFound_returns404

Me:

- me_missingHeader_returns401
- me_userFound_returnsDto
- me_userNotFound_returns404

Premium users:

- Method present; dedicated test commented out (acceptable).

Class: UserSignupDeleteController

Public methods

Method	Tested ?
ResponseEntity<?> getAllUsers()	✓
ResponseEntity<?> getUserById(Long id)	✓
ResponseEntity<?> getByUsername(String username)	✓
String ping()	✓
ResponseEntity<?> deleteUser(Long id)	✓

Tests:

- getAllUsers_empty_returnsEmptyList
- getUserById_found_returnsDto
- getUserById_notFound_returns404
- getByUsername_found_returnsDto
- getByUsername_notFound_returns404
- ping_returnsAliveString
- deleteUser_notFound_returns404
- deleteUser_existing_deletesAndReturnsOk

Class: UserServiceImplemented

Public methods

Method	Tested ?
User signup(String email, String username, String password, String firstName, String lastName)	✓
Optional<User> getById(Long id)	✓
User updateProfile(Long id, String email, String username, String firstName, String lastName, String avatarUrl)	✓
Optional<User> login(String email, String rawPassword)	✓
Optional<User> findByEmail(String email)	✓
Optional<User> findByUsername(String username)	✓
User save(User user)	✓
List<User> findPremiumUsers()	✓

Tests:

Signup:

- signup_success_createsUserWithHashedPasswordAndDefaults
- signup_throwsWhenEmailMissing
- signup_throwsWhenUsernameMissing
- signup_throwsWhenPasswordMissing
- signup_throwsWhenEmailAlreadyExists
- signup_throwsWhenUsernameAlreadyExists

GetById:

- getById_delegatesToRepository

UpdateProfile:

- updateProfile_throwsWhenUserNotFound
- updateProfile_throwsWhenNewEmailAlreadyInUse
- updateProfile_throwsWhenNewUsernameAlreadyInUse
- updateProfile_updatesFieldsAndSaves

Login:

- login_returnsEmptyWhenEmailNull
- login_returnsEmptyWhenPasswordNull
- login_returnsEmptyWhenUserNotFound
- login_returnsEmptyWhenPasswordDoesNotMatch
- login_success_returnsUser

Finders:

- findByEmail_null_returnsEmpty
- findByEmail_nonNull_trimsAndDelegates
- findByUsername_null_returnsEmpty
- findByUsername_nonNull_trimsAndDelegates

Save:

- save_delegatesToRepository

Premium:

- findPremiumUsers_delegatesToRepository

Class: User (entity)

Public API

Feature	Tested ?
setEmail (valid + invalid branches)	✓
setUsername (valid + invalid branches)	✓
setPasswordHash (valid + invalid branches)	✓
setPassword (valid + invalid)	✓
setFirstName (trim, blank → clear)	✓
setLastName (trim, blank → clear)	✓
setAvatarUrl (trim, blank → clear)	✓
setCreatedAt (null or value)	✓

setUpdatedAt	✓
setRole	✓
setActive	✓
setPremiumUser	✓
setId	✓

Tests:

- setEmail_valid_trimsAndSets
- setEmail_null_throws
- setEmail_blank_throws
- setEmail_invalidFormat_throws
- setUsername_valid_trimsAndSets
- setUsername_null_throws
- setUsername_blank_throws
- setUsername_tooShort_throws
- setUsername_tooLong_throws
- setPasswordHash_null_throws
- setPasswordHash_blank_throws
- setPasswordHash_valid_setsValue
- setPassword_null_throws
- setPassword_blank_throws
- setPassword_valid_setsValue
- setFirstName_blank_clearsValue

- `setFirstName_nonBlank_trims`
- `setLastName_blank_clearsValue`
- `setLastName_nonBlank_trims`
- `setAvatarUrl_blank_clearsValue`
- `setAvatarUrl_nonBlank_trims`
- `setCreatedAt_null_setsNow`
- `setCreatedAt_nonNull_setsGivenValue`
- `setUpdatedAt_nonNull_setsGivenValue`
- `setRole_setsGivenValue`
- `setActive_updatesFlag`
- `setPremiumUser_updatesFlag`
- `setId_setsId`

Class: Role (enum)

Public API

Component

Tested?

Enum
usage

Indirectly covered (UserTest asserts roles, APIs return string names)

Enums typically do not require unit tests.

Class: UserRepository

Coverage

Method	Tested?
findByld, existsByld, deleteByld	✓ (indirect via controllers/services)
findByEmailIgnoreCase, findByUsernameIgnoreCase	✓ (indirect)
existsByEmail, existsByUsername	✓ (service tests)
findAllByIsPremiumTrueOrderByldAsc	✓ (service tests)

Repositories are tested indirectly, as expected.